AI Workload Security Assessment

Implementation Solution Guide

Version 1.0.0 · 2026-07-11

Contents

Executive Summary — AI Workload Security Assessment 5
- What you get . . . 5
- What this delivers . . . 5
- The standard deployment model . . . 6
- Outcome . . . 6
- Scope boundary . . . 6

1. Solution Overview — AI Workload Security Assessment 6
- What this assessment is . . . 6
- The review areas . . . 7
- How the assessment works . . . 7
- What you can control . . . 8
- Scope — what's in . . . 8
- Scope — what's out (be honest with buyers) . . . 8
- Authentication and connections used . . . 8
- The standard deployment model . . . 9

2. Requirements — AI Workload Security Assessment 9
- R1. Azure **Reader** at every subscription / resource group hosting AI workloads . . . 9
- R2. Security Reader in Entra (for identity / RBAC review) . . . 9
- R3. Read scopes / provider access for the AI resource types . . . 10
- R4. PowerShell 7+ with Az and Microsoft.Graph modules . . . 10
- R5. A defined, documented AI assessment scope . . . 10
- R6. Diagnostics to Log Analytics for the AI resources (for optional KQL corroboration) . . . 10
- R7. A secure destination for the findings report . . . 10
- R8. Awareness of preview features and the paired build solutions . . . 11

Global Requirements — Assessments (category-level) 11
- G1. The workload/tenant being assessed . . . 11
- G2. Read-only assessor access (least privilege) . . . 11
- G3. Microsoft Graph / Az / management API access . . . 11
- G4. Licensing for the products in scope . . . 12
- G5. Scope definition and a point-in-time baseline . . . 12
- G6. Best-practice reference baseline . . . 12
- G7. Output handling and confidentiality . . . 12
- G8. Remediation ownership (for the paired remediation) . . . 12

3. Implementation — AI Workload Security Assessment 13
- Stage 0 — Assess: prepare and validate access (read-only) . . . 13

Stage 1 — Configure: choose the baseline and pilot workload . . . 14
Stage 2 — Pilot: discover and review the pilot workload (read-only) . . . 14
Stage 3 — Validate: confirm the pilot findings are real . . . 15
Stage 4 — Expand: review every AI workload and produce the roadmap . . . 15
Re-run to trend . . . 16
5. Testing — AI Workload Security Assessment 16
Gate 1 — Access is valid and read-only . . . 16
Gate 2 — Discovery found every AI resource . . . 17
Gate 3 — Network posture reconciles with the portal . . . 17
Gate 4 — Content-safety findings are real (not preview false positives) . . . 17
Gate 5 — RAG exposure is confirmed by inspection . . . 17
Gate 6 — (Optional) KQL corroboration . . . 18
Gate 7 — Report is complete and reproducible . . . 18
Gate 8 — Whole-estate expand run . . . 18
6. Troubleshooting — AI Workload Security Assessment 18
An AI resource is missing from discovery . . . 19
A whole review area is empty . . . 19
`Connect-AzAccount` succeeds but no AI resources appear . . . 19
Content-safety script reports a gap for a preview feature . . . 19
RAG index flagged as "not permission-trimmed" but it is . . . 19
Graph calls fail with `Authorization_RequestDenied` . . . 19
KQL query errors on table or column names . . . 20
Report scores aren't reproducible . . . 20
The assessment found a resource with no owner / no governance . . . 20
7. Best Practices — AI Workload Security Assessment 20
Keep it read-only, keep it least-privilege . . . 20
Don't cry wolf on preview features . . . 20
Rank RAG data-exposure highest . . . 20
Keyless and private are the target state . . . 21
Score against a named baseline and record it . . . 21
Pilot before you present . . . 21
Treat the report as sensitive . . . 21
Leave the decisions with the owners . . . 21
Where to go next (paired AI catalog solutions) . . . 21
8. Deliverables Summary — AI Workload Security Assessment 22
Documentation . . . 22

Automation artifacts (all read-only) . . . 23
What the buyer receives . . . 24
Outcome . . . 24
Appendix A — PowerShell Scripts 25

Executive Summary — AI Workload Security Assessment

White-label · Assessments

Teams stand up Azure AI the fast way: create an Azure OpenAI resource, leave the **public endpoint on**, paste the **account key** into an app setting, point a **RAG** pipeline at a SharePoint site, and ship. It demos brilliantly — and it quietly opens a new class of attack surface that traditional security reviews don't cover: **prompt injection**, **data leakage through over-shared RAG indexes**, **over-permissioned identities**, **public model endpoints**, **disabled content safety**, and **ungoverned model use** with no responsible-AI review. This assessment finds those gaps before an attacker or an auditor does. It discovers your Azure AI estate — **Azure OpenAI, Azure AI Foundry, Azure AI Search, and Azure Machine Learning** — reviews it against an **AI security baseline**, and delivers a **prioritized findings report**.

This is a complete, **run-it-yourself** package written for a competent administrator or ML engineer who has used the Azure portal but has never systematically security-reviewed an AI workload — never checked whether an Azure OpenAI resource is keyless and private, whether a search index is permission-trimmed, or whether content-safety filters are actually attached. The assessment is **strictly read-only**: every script observes and reports; **nothing in your environment is changed**.

What you get

• A plain-language **Solution Overview** of the AI attack surface — what makes AI workloads different, the review areas (discovery, network isolation, keyless identity, content safety / prompt shields / groundedness, RAG data exposure and secrets, responsible-AI governance), and how they roll up into a prioritized findings report.

• A **Requirements Guide** with verify / implement-if-missing for every solution-specific prerequisite (the category-wide ones live in the separate **Global Requirements — Assessments**).

• An **Implementation Guide** of precise, ordered steps — validate read-only access, discover every AI resource, review network isolation and keyless identity, check content-safety and groundedness posture, inspect RAG indexes for over-exposure and hard-coded secrets, and surface responsible-AI governance gaps. Pilot on one workload first, validate the findings, then expand.

• Five read-only **PowerShell** automation scripts (Az / Microsoft Graph / Cognitive Services + Search management) — discovery, access validation, network-isolation review, content-safety review, and the prioritized findings report.

• **Testing** and **Troubleshooting** guides so you can trust the findings before you present them.

What this delivers

A **scored security picture of your AI workloads**: an inventory of every Azure OpenAI / AI Foundry / AI Search / Azure ML resource with its risk posture; a **network-isolation** review (public network access, private endpoints, disabled DNS exfiltration paths); a **keyless-identity** review (local-auth/key usage vs managed identity + least-privilege RBAC); a **content-safety** posture (content filters attached, prompt-shield / jailbreak protection, groundedness detection — noting preview features to verify); a **RAG data-exposure** review (search indexes without permission-trimming, over-broad data-source scope, secrets in connection strings); and a **responsible-AI governance** gap list — all sequenced into a **risk-ranked findings report**. A point-in-time snapshot you can re-run as you remediate.

The standard deployment model

Assess → **Configure** → **Pilot** → **Validate** → **Expand** — discover the AI estate, configure the scope and baseline, pilot the review on one workload, validate the findings against the live portals to remove false positives, then expand to every AI workload and produce the report. The assessment supplies the technical evidence; **acceptable-AI-use policy, data-classification, risk-acceptance, and any regulatory determination remain the customer's decision** — made by your trust-and-safety, legal, and security functions, not by this guide.

Outcome

A defensible answer to "is the AI we're building secure?" — an inventory, a ranked list of the prompt-injection, data-leakage, identity, network, and governance gaps that carry the most risk, and a clear hand-off to the fixes — produced by a single administrator, entirely read-only, and re-runnable to prove progress.

Scope boundary

This assessment **reviews Azure AI workloads you build** (Azure OpenAI / AI Foundry / AI Search / Azure ML). It **pairs with, and does not duplicate,** the AI-platform build solutions — **Azure AI Foundry & OpenAI secure deployment (AIFOUNDRY)**, **Enterprise RAG platform (AIRAG)**, **AI Content Safety & Responsible-AI guardrails (AISAFETY)**, and **AI Landing Zone governance (AILZ)** — which *deploy* the secure configurations this assessment checks for. It is **not** Microsoft 365 Copilot governance (that is the Copilot / Purview-AI areas), and it does **not** remediate — the paired AI solutions do that, pilot-first. Cross-referenced, not duplicated.

1. Solution Overview — AI Workload Security Assessment

What this assessment is

This solution is a **security assessment of the Azure AI workloads your teams build** — **Azure OpenAI**, **Azure AI Foundry** (hub/project), **Azure AI Search**, and **Azure Machine Learning**. It **discovers** what AI resources exist, **reviews** each against an AI security baseline, and **prioritizes** the gaps into a risk-ranked findings report. It is not a deployment and not a remediation: every script is **read-only** and changes nothing. The output is evidence — an inventory and a ranked findings list — that tells you where an AI workload is exposed and which gap to close first.

AI workloads matter here because they add attack surface that a conventional cloud security review misses:

- **Prompt injection** — untrusted content (a document, a web page, a user message) that hijacks the model's instructions. Content filters and **prompt shields** exist to blunt it; the question is whether they're turned on.
- **Data leakage through RAG** — a retrieval-augmented-generation app surfaces answers from an index built over your files. If the **index isn't permission-trimmed**, a user can get answers drawn from documents they could never open directly. This is the single most common serious AI finding.

- **Over-permissioned identity and keys** — an Azure OpenAI **account key** is a long-lived, portable credential; **keyless** (managed identity + least-privilege RBAC) is the secure path.
- **Public endpoints** — an AI resource reachable from the internet is a data-exfiltration surface; **private endpoints** and **disabled public network access** close it.
- **Ungoverned model use** — models deployed with no content-filter policy, no responsible-AI review, and no owner.

Reviewing against a **named baseline** makes the findings comparable and defensible run-to-run. *The AI security baseline, the Azure AI resource model, and several safety features evolve quickly and some are in preview — verify current availability, resource types, and portal layout in your tenant when you run the assessment.*

The review areas

- **Discovery.** Enumerate every Azure OpenAI / Cognitive Services (kind `OpenAI` / `AIServices`), AI Foundry hub/project, Azure AI Search service, and Azure ML workspace in scope, with its headline posture.
- **Network isolation.** For each AI resource: is **public network access disabled**? Is there a **private endpoint** on your VNet? Are there routes that still allow public egress/exfiltration? Public-by-default is the norm, so this is where most resources fail.
- **Keyless identity.** Is **local auth (account keys) disabled**? Does the app authenticate with a **managed identity** holding a least-privilege data-plane role (e.g. Cognitive Services OpenAI User; Search Index Data Reader) rather than a shared key? Are there keys in app settings?
- **Content safety, prompt shields, groundedness.** Does each model deployment have a **content-filter (RAI) policy** attached? Are **prompt-shield / jailbreak** and **protected-material** protections enabled? Is **groundedness detection** in use for RAG? *Several of these are preview — verify availability and preview status.* *The acceptable-content threshold itself is the customer's trust-and-safety decision.*
- **RAG data exposure and secrets.** For AI Search: are indexes **permission-trimmed** (security filters / document-level access), or do they expose the whole corpus to every user? How broadly scoped are the data sources? Are there **secrets / keys hard-coded** in connection strings or app settings instead of Key Vault?
- **Responsible-AI governance.** Is there an owner, a documented purpose, a content-filter standard, and a review for each model in use — or are models deployed ad hoc? *The governance policy itself is the customer's; the assessment scores whether controls exist.*

How the assessment works

- **Read-only discovery** via **Az** modules (Az.CognitiveServices, Az.Search, Az.MachineLearningServices, Az.Network, Az.Monitor) and **Microsoft Graph** for identity/RBAC — authenticated as a **least-privilege reader** (Azure Reader + Security Reader), never with write access.
- **Posture rules** encode the AI security baseline (public access disabled, local auth disabled, custom subdomain set, private endpoint present, content filter attached, permission-trimming on indexes, no plaintext secrets) and flag each resource that deviates as a finding.

• **KQL where it adds signal.** Where diagnostics flow to Log Analytics / Sentinel, the guide supplies read-only KQL to corroborate posture (e.g. Azure OpenAI calls that authenticated with a key rather than a token) — evidence, never a change.

• **Ranking.** Findings are tagged with a **risk rating** (impact × exposure) — a public, keyless-disabled resource serving a non-permission-trimmed RAG index over sensitive data ranks Critical — and sequenced into a phased roadmap.

What you can control

• **Scope** — which subscriptions, resource groups, and AI workloads are in the assessment.

• **Baseline** — the AI security baseline you score against (Microsoft AI security guidance + the paired build solutions' target state). *Verify the current baseline.*

• **Output handling** — where the (sensitive) findings report is stored and who may see it.

Scope — what's in

• **Read-only discovery** of Azure OpenAI / AI Foundry / AI Search / Azure ML.

• **Network-isolation** review (public access, private endpoints).

• **Keyless-identity** review (local auth, managed identity + least-privilege RBAC, plaintext secrets).

• **Content-safety / prompt-shield / groundedness** posture (preview features flagged).

• **RAG data-exposure / permission-trimming** review and **secrets** review.

• **Responsible-AI governance** gap list, rolled into a risk-ranked findings report.

Scope — what's out (be honest with buyers)

• **This does not remediate.** It observes and scores. The paired AI solutions deploy the fixes: **AIFOUNDRY** (private, keyless Azure OpenAI + Foundry), **AIRAG** (permission-trimmed RAG on AI Search), **AISAFETY** (content safety / responsible-AI guardrails), **AILZ** (AI landing-zone governance). Cross-referenced, not duplicated.

• **Not Microsoft 365 Copilot governance.** Governing Copilot over M365 data is the **Copilot** / **Purview-AI** areas; this assesses AI apps your teams build.

• **Not a model red-team / jailbreak engagement.** The assessment checks that safety controls are configured; adversarial prompt testing of a live model is a separate discipline.

• **No acceptable-use, data-classification, or regulatory determination.** The assessment states each gap and its risk; **what content is acceptable, what data is sensitive, whether to accept a risk, and any regulatory mapping are the customer's decisions** — trust-and-safety, legal, and security own them.

Authentication and connections used

• **Az PowerShell** signed in as **Reader** at the Azure scope — for AI resource, network, and diagnostics discovery (Global Requirements G2, G3).

• **Microsoft Graph** (read scopes) signed in as **Security Reader** — for identity / RBAC / managed-identity role assignments (G2, G3).

• **Microsoft Sentinel / Log Analytics** read access where present — for the corroborating KQL (optional).

• No write scopes are requested or used — the read-only guarantee (G2).

The standard deployment model

> ***Assess the AI estate (discover every AI resource and its posture) → configure the scope and AI security baseline → pilot the review on one AI workload → validate the findings against the live Azure portal to remove false positives → expand to every AI workload and produce the risk-ranked report.*** *The repeatable pattern:* ***Assess → Configure → Pilot → Validate → Expand*** *— entirely read-only, re-runnable to trend progress, with remediation handed to the paired AI build solutions under change control.*

2. Requirements — AI Workload Security Assessment

The category-wide prerequisites (the target tenant/subscription, read-only assessor access, Graph/Az API access, licensing, scope/baseline, output handling, remediation ownership) are published once in **Global Requirements — Assessments** — read that first. This page lists only the prerequisites **specific to assessing Azure AI workloads**. Each item: **Description / Why required / How to verify / How to implement if missing.**

> *Everything here is read-only. If satisfying an item seems to need write/Contributor access, you have over-scoped the identity — an AI security review reads configuration, it never changes a resource.*

R1. Azure Reader at every subscription / resource group hosting AI workloads

Description: The Azure RBAC **Reader** role at each subscription (or resource group) that contains Azure OpenAI, AI Foundry, AI Search, or Azure ML resources in scope. **Why required:** Discovery, network-isolation, and content-safety reviews read the AI resources through Azure Resource Manager; without Reader they return nothing and the workload appears (falsely) absent. **How to verify:** `portal.azure.com` → the subscription/RG → **Access control (IAM)** → **View my access** → confirm **Reader**. `AIWLSEC-Validate.ps1` also confirms `Connect-AzAccount` and lists AI resources found. **How to implement if missing:** Have an Owner/User Access Administrator assign **Reader** at the highest common scope covering the AI workloads. Do **not** request Contributor for the assessment.

R2. Security Reader in Entra (for identity / RBAC review)

Description: The **Security Reader** (or Global Reader) directory role, to read managed-identity role assignments and app registrations tied to the AI workloads. **Why required:** The keyless-identity review inspects which identities hold data-plane roles on the AI resources and whether app registrations carry secrets; that needs directory read access. **How to verify:** Entra admin center → **Roles and administrators** → confirm **Security Reader** / **Global Reader**. `AIWLSEC-Validate.ps1` confirms the Graph read scopes. **How to implement if missing:** Have a Privileged Role Administrator assign **Security**

Reader (read-only).

R3. Read scopes / provider access for the AI resource types

Description: Reader-level access to the resource providers the scripts query — **Microsoft.CognitiveServices** (Azure OpenAI / AI Services), **Microsoft.Search** (AI Search), **Microsoft.MachineLearningServices** (Azure ML), **Microsoft.Network** (private endpoints), and **Microsoft.Insights** (diagnostics). Graph read scopes such as `Directory.Read.All` for RBAC. *Verify the current provider/API and Graph scope set.* **Why required:** Each review area calls a specific provider; a missing provider registration or scope makes that area silently empty. **How to verify:** `Get-AzResourceProvider -ProviderNamespace Microsoft.CognitiveServices` shows Registered; `Connect-MgGraph -Scopes "Directory.Read.All"` succeeds. `AIWLSEC-Validate.ps1` reports coverage. **How to implement if missing:** Ensure the providers are registered on the subscription (they are once the resources exist) and consent the read-only Graph scopes.

R4. PowerShell 7+ with Az and Microsoft.Graph modules

Description: PowerShell 7+ with **Az.Accounts / Az.Resources / Az.CognitiveServices / Az.Search / Az.MachineLearningServices / Az.Network / Az.Monitor** and **Microsoft.Graph**. *Verify current versions.* **Why required:** All five scripts are PowerShell and depend on these modules; version drift is the most common cause of empty output. **How to verify:** `$PSVersionTable.PSVersion` ≥ 7; `Get-Module Az.CognitiveServices,Az.Search -ListAvailable`. **How to implement if missing:** `Install-Module Az -Scope CurrentUser` and `Install-Module Microsoft.Graph -Scope CurrentUser`. Some ML/Search cmdlets evolve — verify the current module names for your subscription's resource types.

R5. A defined, documented AI assessment scope

Description: A written scope — which subscriptions/RGs, which AI workloads, and the AI security baseline you score against — agreed before the run. **Why required:** Scope keeps the review bounded and the findings comparable run-to-run. **How to verify:** The scope document exists and names the workloads and baseline; the report records scope and run date. **How to implement if missing:** Agree the scope with the AI/security owner. Start with one workload for the pilot, then expand.

R6. Diagnostics to Log Analytics for the AI resources (for optional KQL corroboration)

Description: Azure OpenAI / AI Search diagnostic settings streaming to a **Log Analytics** workspace (or Sentinel), if you want to corroborate posture with the supplied read-only KQL. **Why required:** The KQL corroboration (e.g. key-based auth still in use) needs the logs; without them the assessment still scores from configuration but loses the live cross-check. **How to verify:** The AI resource → **Diagnostic settings** shows a workspace destination; you can query the relevant table. *Verify current table/category names.* **How to implement if missing:** Note absent diagnostics as a finding (you can't observe what you don't log), or skip the KQL — it is optional, not required, for the findings.

R7. A secure destination for the findings report

Description: A restricted-access location for the exported findings, which map your AI attack surface. **Why required:** The report details where an AI workload is exposed; it must be handled as sensitive. **How to verify:** A restricted SharePoint/Teams site or secured share exists, access limited to the security/AI owners. **How to implement if missing:** Provision a restricted destination before running; share on a need-to-know basis. *Risk-acceptance of any finding remains the customer's decision.*

R8. Awareness of preview features and the paired build solutions

Description: An understanding that several AI safety controls (prompt shields, groundedness detection, protected-material detection, asynchronous filtering) may be in **preview**, and that the *fixes* for findings are the paired AI build solutions (AIFOUNDRY / AIRAG / AISAFETY / AILZ). **Why required:** The assessment must not assert a control is "missing" when it is simply unavailable/preview in your region; and findings need an owner and a remediation path. **How to verify:** The report flags preview-dependent findings with "*verify current availability*"; the remediation column names the paired solution. **How to implement if missing:** *Verify current availability and preview status* for any safety feature before treating its absence as a hard finding; assign remediation owners and reference the paired solutions.

Global Requirements — Assessments (category-level)

These requirements apply to every solution in the Assessments category (Security Optimization Assessments, remediation playbooks, and cross-cutting security assessments). Published once at the category level so listings stay small. Confirm these before running any assessment. Each entry: Description / Why / Verify / Implement-if-missing.

> *An assessment observes and scores; it does not change your tenant. Every assessment script here is **read-only** (the paired remediation work is separate and pilot-first). Scope, risk-acceptance, and remediation-priority decisions belong to the customer's security/compliance owners — the assessment supplies the evidence, not the authorization.*

G1. The workload/tenant being assessed

Description: An active Microsoft 365 / Entra tenant and/or Azure subscription(s) hosting the products in scope. Why: There is nothing to assess without the target environment. Verify: Sign in to the relevant portal(s) (`security.microsoft.com`, `portal.azure.com`) as an administrator. Implement: Ensure the target tenant/subscription exists and the products under assessment are deployed (even partially — assessing coverage gaps is the point).

G2. Read-only assessor access (least privilege)

Description: A read-only role sufficient to enumerate configuration — e.g. Security Reader, Global Reader, and/or Reader at the Azure scope. Why: An assessment must never require write access; read-only proves it changes nothing. Verify: Entra -> `Roles and administrators`; confirm Security Reader / Global Reader (and Azure Reader where cloud resources are assessed). Implement: Assign the read-only roles; do not grant write/admin roles for the assessment itself.

G3. Microsoft Graph / Az / management API access

Description: Microsoft Graph (Security), Azure Resource Manager, and product-specific management APIs reachable with the assessor identity. Why: The assessment scripts collect configuration and posture via these APIs. Verify: `Connect-MgGraph` (read scopes) and `Connect-AzAccount` succeed; the assessment discovery script runs. *Verify current API/permission scopes.* Implement: Consent the read-only Graph scopes; ensure network access to the Microsoft endpoints.

G4. Licensing for the products in scope

Description: The licenses/plans for the products being assessed (e.g. Defender for Endpoint/Identity/Office 365/Cloud Apps plans, Defender for Cloud plans). Why: Some posture/configuration is only present (and only assessable) when the plan is enabled; the assessment flags plan gaps too. Verify: Confirm the relevant plans in the admin/billing portal. *Verify current plan/feature mapping.* Implement: Note unlicensed products as findings (a coverage gap) rather than skipping them silently.

G5. Scope definition and a point-in-time baseline

Description: A defined assessment scope (which products, subscriptions, and users) and the understanding that results are a point-in-time snapshot. Why: Scope keeps the assessment bounded and the score comparable; posture drifts, so results are dated. Verify: The scope is documented; the assessment records the run date. Implement: Agree the scope before running; re-run periodically to trend the score.

G6. Best-practice reference baseline

Description: The best-practice / benchmark the assessment scores against (Microsoft security baselines, Secure Score, CIS/Zero-Trust as applicable). Why: A score is only meaningful against a reference; the assessment states which baseline it uses. Verify: The reference baseline is named in the findings. *Baselines evolve — verify against current Microsoft guidance.* Implement: Use the documented baseline; note where organizational policy intentionally deviates.

G7. Output handling and confidentiality

Description: A place to store the findings report and an understanding that it may contain sensitive posture detail. Why: Assessment output is a roadmap of your weaknesses; it must be handled as sensitive. Verify: A secure destination for the report exists; access is restricted. Implement: Store findings securely; share on a need-to-know basis. *Risk-acceptance of any finding is the customer's decision.*

G8. Remediation ownership (for the paired remediation)

Description: Named owners for acting on findings (the remediation is separate, pilot-first work). Why: An assessment only creates value when someone owns closing the gaps. Verify: Remediation owners are identified; the Remediation Playbook solution is available for the fixes. Implement: Assign owners; use the Remediation Playbook (pilot-first, -WhatIf-gated) to close findings and re-score.

The deployment model (all solutions): Assess -> Configure -> Pilot -> Validate -> Expand.

3. Implementation — AI Workload Security Assessment

This is the exact, ordered procedure. It is **read-only** end to end — you enumerate and score, you never change a resource. Follow the standard model: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**. Run the **pilot** on one AI workload first, confirm the findings are real, then expand to every AI workload.

Naming convention. Every artifact you create uses `<Purpose> – <Scope> – <Stage>`. Reuse the exact names in later steps, testing, and the report:

- Scope document: `AIWorkloadSec – Estate – Scope`
- Output folder: `AIWorkloadSec – Estate – Findings`
- Pilot run label: `AIWorkloadSec – ChatbotPilot – Pilot`
- Full run label: `AIWorkloadSec – Estate – Full`
- Roadmap deliverable: `AIWorkloadSec – Estate – Roadmap`
- Saved KQL query: `AIWorkloadSec – KeyAuth – Signal`

Stage 0 — Assess: prepare and validate access (read-only)

Step 0.1 — Write the scope document

1. Create `AIWorkloadSec – Estate – Scope`. Record: the subscriptions/resource groups in scope, the AI

workloads in scope (name each Azure OpenAI / AI Foundry / AI Search / Azure ML resource you know of), the **AI security baseline** you score against (Microsoft AI security guidance + the paired build solutions' target state — *verify the current baseline*), and the run date (the score is point-in-time, G5).

Step 0.2 — Create the output folder

1. Create a restricted folder `AIWorkloadSec – Estate – Findings` in your secure destination (R7). All

exports land here.

Step 0.3 — Install modules and validate read-only access

1. In **PowerShell 7+**: `Install-Module Az -Scope CurrentUser` and

`Install-Module Microsoft.Graph -Scope CurrentUser` if not present (R4).

1. Connect read-only:

- `Connect-AzAccount` (then `Set-AzContext -SubscriptionId <sub>` per in-scope subscription).
- `Connect-MgGraph -Scopes "Directory.Read.All","Policy.Read.All"` *(verify the current least-privilege scope set)*.

1. Run `AIWLSEC-Validate.ps1`. It confirms PowerShell/module versions, that Azure and Graph are

connected, that your identity is **read-only** (Azure Reader + Security Reader, no Contributor/write), that the AI resource providers are registered, and lists the **AI resources it can see** in scope. **Do not proceed until Validate passes** — a missing role makes a whole review area silently empty.

Stage 1 — Configure: choose the baseline and pilot workload

1. In `AIWorkloadSec – Estate – Scope`, confirm and record the **AI security baseline** — the report must

name what it scored against (G6).

1. Pick one **representative AI workload** for the pilot (e.g. a customer-facing chatbot with a RAG index) and

label it `AIWorkloadSec – ChatbotPilot – Pilot`. The pilot proves the method and surfaces false positives before the full run.

Stage 2 — Pilot: discover and review the pilot workload (read-only)

Step 2.1 — AI estate discovery

1. Run `AIWLSEC-Discovery.ps1 -SubscriptionIds <pilot sub> -OutputFolder "<...>\AIWorkloadSec – Estate – Findings"`.

It enumerates, read-only, every:

• **Azure OpenAI / Cognitive Services** account (kind `OpenAI` / `AIServices`) — public access, local-auth

(keys) state, custom subdomain, model deployments and content-filter policies, private endpoints, diagnostics.

• **Azure AI Foundry** hub/project (*verify current resource types*).

• **Azure AI Search** service — public access, API-key vs RBAC (`disableLocalAuth`/`authOptions`), indexes,

and whether data sources indicate a RAG pattern.

• **Azure ML** workspace — public access, managed VNet, associated storage/Key Vault posture.

1. It writes a per-resource posture CSV/JSON to the output folder.

Step 2.2 — Network isolation review

1. Run `AIWLSEC-NetworkIsolation.ps1 -SubscriptionIds <pilot sub> -OutputFolder "<...>\AIWorkloadSec – Estate – Findings"`.

For every AI resource it reports: **public network access** (Enabled = finding), **private endpoint** present/absent, and whether a private DNS path exists — the highest-frequency AI findings live here.

Step 2.3 — Content-safety / responsible-AI posture review

1. Run `AIWLSEC-ContentSafety.ps1 -SubscriptionIds <pilot sub> -OutputFolder "<...>\AIWorkloadSec – Estate – Findings"`.

For each Azure OpenAI model deployment it reports whether a **content-filter (RAI) policy** is attached and the default filter state, and flags **prompt-shield / jailbreak / groundedness** protections to verify. *Several of these are preview — the script marks them "verify availability", it does not assert a gap where the feature is simply unavailable in your region.*

Step 2.4 — RAG data-exposure and secrets review (manual + scripted)

1. From the discovery output, for each **AI Search** index that backs a RAG app, confirm in the portal

(`portal.azure.com` → the Search service → **Indexes** / **Search explorer**, and the app's data-access design) whether the index is **permission-trimmed** (security filters / document-level access) or exposes the whole corpus to every caller. Record the answer as a finding — a non-trimmed index over sensitive data is Critical.

1. Check the app's configuration and any Foundry/Search **connections** for **hard-coded keys / secrets** in

connection strings or app settings (they belong in Key Vault). Record any found. *Data-classification of what the index contains is the customer's decision; the assessment scores whether the access control exists.*

Step 2.5 — (Optional) corroborate with KQL

1. If diagnostics flow to Log Analytics/Sentinel (R6), run the read-only query saved as

`AIWorkloadSec – KeyAuth – Signal` (in Testing) to see whether Azure OpenAI calls are still authenticating with a **key** rather than a token — live evidence behind the keyless finding.

Step 2.6 — Score and rank the pilot

1. Run `AIWLSEC-Report.ps1 -InputFolder "<...>\AIWorkloadSec – Estate – Findings" -Label "AIWorkloadSec – ChatbotPilot – Pilot"`.

It rolls the discovery + network + content-safety outputs (and any RAG/secrets findings you recorded) into a per-resource posture, tags each finding with a **risk rating**, and emits a draft risk-ranked report.

Stage 3 — Validate: confirm the pilot findings are real

1. For each **High/Critical** finding, open the live Azure portal blade and confirm:

• Network → the AI resource → **Networking** (public access + private endpoints).

• Keyless → the resource → **Keys and Endpoint** / **Identity**, and **Access control (IAM)** for the

managed-identity role.

• Content safety → Azure AI Foundry / Azure OpenAI → the deployment's **content filter**.

• RAG → the AI Search service → **Indexes** and the app's access design.

1. Mark any confirmed false positive with the reason (e.g. a preview feature not yet GA in the region, or a

compensating control the script can't see) — **do not silently delete findings**.

1. If a review area returned empty, re-check the matching role/scope (R1–R3) — a common cause.

Stage 4 — Expand: review every AI workload and produce the roadmap

1. Re-run **Steps 2.1–2.6** with the full scope, labelled `AIWorkloadSec – Estate – Full` — point the

scripts at all in-scope subscriptions/RGs and review every AI resource.

1. Run `AIWLSEC-Report.ps1 ... -Label "AIWorkloadSec – Estate – Full"` to produce the final

risk-ranked findings report.

1. Build the roadmap `AIWorkloadSec – Estate – Roadmap` in three phases, mapping each item to the paired

AI build solution that fixes it:

• **Phase 1 — Quick wins:** disable public network access, attach a content-filter policy, disable local

auth on resources already keyless-capable → **AIFOUNDRY** / **AISAFETY**.

• **Phase 2 — Foundational:** private endpoints + DNS, managed-identity + least-privilege RBAC, move

secrets to Key Vault → **AIFOUNDRY**; permission-trim RAG indexes → **AIRAG**.

• **Phase 3 — Strategic:** responsible-AI governance program, prompt-shield/groundedness rollout,

AI landing-zone guardrails → **AISAFETY** / **AILZ**.

1. Store the report and roadmap in `AIWorkloadSec – Estate – Findings` (restricted). Present the executive

summary (top AI risks) to leadership and the technical findings to the AI/workload owners.

> ***Determinations that are NOT yours to make in this guide:*** *what content is *acceptable*, what data is *sensitive*, whether to *accept* a given risk, and any* ***regulatory / legal*** *mapping of a finding belong to your trust-and-safety, legal, and security functions. The assessment supplies the evidence and a recommended sequence; the decisions are the customer's.*

Re-run to trend

Re-run the whole assessment after each remediation wave with the same scope and baseline to **trend** the AI security posture and prove the gaps are closing.

5. Testing — AI Workload Security Assessment

Testing an assessment means proving the **findings are trustworthy** before anyone acts on them: the access is truly read-only, discovery found every AI resource, the posture reconciles with the live portal, and the report is reproducible. Work the gates in order; each has an explicit **pass/fail**. All tests are read-only.

Scripts referenced (all read-only): `AIWLSEC-Validate.ps1`, `AIWLSEC-Discovery.ps1`, `AIWLSEC-NetworkIsolation.ps1`, `AIWLSEC-ContentSafety.ps1`, `AIWLSEC-Report.ps1`.

Gate 1 — Access is valid and read-only

Run `AIWLSEC-Validate.ps1`.

• **Pass:** PowerShell ≥ 7; required Az + Graph modules present; Azure and Graph connected; identity resolves

to **Azure Reader** + **Security Reader** with **no Contributor/write role**; AI resource providers registered; in-scope AI resources listed.

• **Fail:** Any missing module/role, or a write role detected. Fix per Requirements R1–R4 — a missing role makes a review area falsely empty; a write role breaks the read-only guarantee.

Gate 2 — Discovery found every AI resource

Run `AIWLSEC-Discovery.ps1` on the pilot scope, then compare against `portal.azure.com` → **All resources** filtered to Cognitive Services / Search / Machine Learning.

• **Pass:** Every AI resource visible in the portal for the scope appears in the discovery output, and the per-resource posture CSV/JSON lands in `AIWorkloadSec – Estate – Findings`.

• **Fail:** A resource is missing from the output. Usually **Reader** isn't assigned on that resource's subscription/RG (R1), or the provider isn't registered (R3). Fix and re-run.

Gate 3 — Network posture reconciles with the portal

Run `AIWLSEC-NetworkIsolation.ps1`, then open a sample AI resource → **Networking**.

• **Pass:** The public-network-access state and private-endpoint presence the script reports **match** the portal for the sampled resources.

• **Fail:** They diverge. Confirm you're reading the same resource/tenant and that Az.Network is current; re-run.

Gate 4 — Content-safety findings are real (not preview false positives)

Run `AIWLSEC-ContentSafety.ps1`, then open a model deployment → **content filter** in Azure AI Foundry / Azure OpenAI.

• **Pass:** Each "no content filter attached" finding is confirmed in the portal; each preview-dependent item

(prompt shields, groundedness) is flagged "*verify availability*" rather than asserted as a hard gap.

• **Fail:** The script asserts a gap for a feature that is simply **preview/unavailable** in your region. Downgrade it to a "verify availability" note — do not report an unavailable feature as a missing control.

Gate 5 — RAG exposure is confirmed by inspection

For each RAG-backing AI Search index, inspect permission-trimming per Implementation Step 2.4.

• **Pass:** Each "index not permission-trimmed" finding is confirmed by looking at the index's security filtering / document-level access and the app's access design.

• **Fail:** The index actually *is* trimmed by a mechanism the script couldn't see (app-side security filter). Record as a validated false positive with the reason.

Gate 6 — (Optional) KQL corroboration

If diagnostics flow to Log Analytics/Sentinel (R6), run the saved query `AIWorkloadSec – KeyAuth – Signal`:

```
// Read-only. Azure OpenAI / Cognitive Services requests still authenticating with a key rather than an
// Entra token — corroborates the "local auth (keys) enabled" finding. *Verify table/column names for your
// workspace and the current diagnostic category.*
AzureDiagnostics
| where ResourceProvider == "MICROSOFT.COGNITIVESERVICES"
| where TimeGenerated > ago(7d)
| extend AuthType = tostring(properties_authType_s)
| summarize Requests = count() by AuthType, Resource
| order by Requests desc
```

- **Pass:** The query runs read-only and either confirms key-based auth is still occurring (supports the

finding) or shows token-only auth (note as a compensating signal).

- **Fail:** Query errors on table/column names — adjust to your workspace schema (*verify current schema*) or

skip; KQL corroboration is optional and does not gate the findings.

Gate 7 — Report is complete and reproducible

Run `AIWLSEC-Report.ps1` for the labelled run.

- **Pass:** The report lists every AI resource with its posture, tags each finding Critical/High/Medium/Low,

names the **AI security baseline and run date** in the header, flags preview-dependent items, and maps each finding to a paired remediation solution (AIFOUNDRY/AIRAG/AISAFETY/AILZ); re-running on the same inputs produces the same result.

- **Fail:** Missing findings, no baseline named, or non-reproducible output. Confirm all Stage-2 outputs are

present in the input folder and re-run.

Gate 8 — Whole-estate expand run

Repeat Gates 2–7 with the full scope labelled `AIWorkloadSec – Estate – Full`.

- **Pass:** All gates pass at full scope; the roadmap `AIWorkloadSec – Estate – Roadmap` sequences findings

into Phase 1/2/3 and each item names its paired build solution.

- **Fail:** A resource that passed at pilot scope now fails at full scope — usually a subscription/RG where

Reader wasn't assigned. Fix the coverage (R1, R5) and re-run.

> *Passing all gates means the findings are defensible. It does **not** authorize any change — remediation is the paired AI build solutions' job, pilot-first and change-controlled, and acceptable-use / risk-acceptance decisions are the customer's.*

6. Troubleshooting — AI Workload Security Assessment

Common failures when running the assessment, as **symptom** → **cause** → **fix**. Everything here stays read-only — no fix below changes an AI resource.

An AI resource is missing from discovery

Symptom: `AIWLSEC-Discovery.ps1` returns some AI resources but not one you can see in the portal. **Cause:** Azure **Reader** isn't assigned on that resource's subscription/RG (R1), or the resource provider isn't registered (R3), or your context is set to a different subscription. **Fix:** Run `AIWLSEC-Validate.ps1` to confirm the visible subscriptions and providers; assign **Reader** at the missing scope, `Set-AzContext` to the right subscription, and re-run. Never assume "not found" means "doesn't exist" until access is confirmed.

A whole review area is empty

Symptom: Discovery works but the network-isolation or content-safety review returns nothing. **Cause:** The specific module or provider for that area is missing/old — network needs **Az.Network**, content safety reads Cognitive Services deployments; version drift is common (R4). **Fix:** `Get-Module Az.Network,Az.CognitiveServices -ListAvailable`; update with `Install-Module Az`. Re-run after confirming the modules and the Reader role for those resources.

`Connect-AzAccount` succeeds but no AI resources appear

Symptom: Azure connects, but the AI estate looks empty. **Cause:** Wrong subscription context, or Reader assigned at a different scope than where the AI resources live. **Fix:** `Get-AzSubscription` and `Get-AzContext`; `Set-AzContext -SubscriptionId <the AI sub>`. Pass the correct `-SubscriptionIds` to the scripts. Re-run.

Content-safety script reports a gap for a preview feature

Symptom: The script flags "no prompt shield / no groundedness detection" but you know it's not yet GA in your region. **Cause:** Several AI safety controls are **preview** or region-limited; absence ≠ misconfiguration. **Fix:** The script marks these "verify availability" by design — treat them as a note, not a hard finding. *Verify current availability and preview status* before reporting a gap (Best Practices, Testing Gate 4).

RAG index flagged as "not permission-trimmed" but it is

Symptom: The assessment flags an AI Search index as exposing the whole corpus, but the app enforces document-level access. **Cause:** Permission-trimming can be enforced **app-side** (a security filter the app injects at query time) which the resource-level scan can't see. **Fix:** Confirm by inspection (Implementation Step 2.4). If it's genuinely trimmed, record a **validated false positive** with the mechanism noted — don't delete it silently. If trimming is only in one code path, the finding may still stand for the others.

Graph calls fail with `Authorization_RequestDenied`

Symptom: The identity/RBAC portion errors on Graph. **Cause:** The **Security Reader** role or the `Directory.Read.All` scope isn't granted (R2, R3). **Fix:** Assign **Security Reader**; re-run `Connect-MgGraph -Scopes "Directory.Read.All"` and consent. Never substitute a write scope.

KQL query errors on table or column names

Symptom: The `AIWorkloadSec – KeyAuth – Signal` query fails with an unknown table/column. **Cause:** Diagnostic category and schema for Cognitive Services vary and evolve; the resource may log to a resource-specific table rather than `AzureDiagnostics`. **Fix:** Adjust the table/column names to your workspace and the current diagnostic category (*verify current schema*), or skip it — the KQL is **optional** corroboration and does not gate the findings.

Report scores aren't reproducible

Symptom: Re-running `AIWLSEC-Report.ps1` yields different results on the "same" data. **Cause:** The input folder changed between runs, or pilot and full-estate outputs were mixed. **Fix:** Ensure all Stage-2 outputs for the run label are present and complete; keep `... – Pilot` and `... – Full` outputs in clearly labelled subfolders. Re-run.

The assessment found a resource with no owner / no governance

Symptom: A model deployment has no content-filter policy, no documented owner, and no review. **Cause:** This is a **finding, not an error** — ungoverned model use is exactly what the assessment surfaces. **Fix:** Record it and map it to **AISAFETY** (content safety / responsible-AI) for remediation. Assigning an owner and a governance standard is the customer's decision, done under change control — not part of this read-only assessment.

7. Best Practices — AI Workload Security Assessment

Field-proven guidance for running an AI workload security assessment that people trust and act on — plus where to go next to close the gaps.

Keep it read-only, keep it least-privilege

- Run the assessment as **Azure Reader + Security Reader** and let `AIWLSEC-Validate.ps1` prove it. An AI

security review that could change a resource is no longer an assessment.

- Consent only **read** Graph scopes. If a script seems to need write access, treat it as a bug.

Don't cry wolf on preview features

- Several AI safety controls (prompt shields, groundedness detection, protected-material detection,

asynchronous filtering) may be **preview** or **region-limited**. *Verify current availability before treating absence as a hard finding.* Report an unavailable feature as "verify availability", not as a missing control — false alarms erode trust in the whole report.

Rank RAG data-exposure highest

- A RAG index that **isn't permission-trimmed** over sensitive data is the most damaging common AI finding:

it silently hands users answers from documents they can't open. Confirm permission-trimming by inspection (the script points you at it) and rank it Critical when the corpus is sensitive.

• *What data is sensitive is the customer's classification decision* — the assessment scores whether the

access control exists, not what should be protected.

Keyless and private are the target state

• Flag every AI resource with **public network access enabled** and every one with **local auth (keys)

enabled**. **The secure target —** private endpoints + disabled public access + managed identity + least-privilege RBAC** — is exactly what the paired build solutions deploy. Point findings at them.

• Flag any **secret/key hard-coded** in a connection string or app setting; it belongs in Key Vault.

Score against a named baseline and record it

• Name the **AI security baseline** and the **run date** in the report header so the findings are comparable

and defensible. *Baselines and Azure AI resource models evolve — verify each run.*

Pilot before you present

• Run **one AI workload** first and **validate High/Critical findings against the live portal** before the

full run. Record any confirmed false positive with the reason rather than deleting it.

Treat the report as sensitive

• The findings map your AI attack surface. Store it in the restricted `AIWorkloadSec – Estate – Findings`

location and share on a need-to-know basis.

Leave the decisions with the owners

• The assessment states each gap and its risk and recommends a sequence. **Acceptable-AI-use policy,

data-classification, risk-acceptance, and any regulatory / legal mapping are the customer's decisions** — trust-and-safety, legal, and security own them, not this guide or its scripts.

Where to go next (paired AI catalog solutions)

This assessment is the **map**; these solutions do the **build/fix** (each pilot-first, `what-if`/approval-gated):

• **Azure AI Foundry & OpenAI secure enterprise deployment (AIFOUNDRY)** — deploys the private, keyless,

quota-bounded, content-filtered Azure OpenAI + Foundry platform the network/identity/content-safety findings call for.

- **Enterprise RAG platform on Azure AI Search (AIRAG)** — builds the **permission-trimmed** RAG index that

closes the data-exposure findings.

- **AI Content Safety & Responsible-AI guardrails (AISAFETY)** — implements the content filters, prompt

shields, and responsible-AI governance the safety/governance findings call for.

- **AI Landing Zone governance & private networking (AILZ)** — the org-wide guardrails, policy, and networking

that keep new AI workloads secure by default.

- **Cloud Adoption Security Assessment (CLDSEC)** — the sibling whole-estate assessment, if you also need the

broader identity/endpoint/data/infra/network picture around these AI workloads.

Close the top-ranked findings with the paired solutions, then **re-run this assessment** to prove the posture moved.

8. Deliverables Summary — AI Workload Security Assessment

Documentation

Document	Purpose
Executive Summary	The business case and what's in the box — a scored security picture of your Azure AI workloads and a prioritized findings report
1. Solution Overview	What the assessment is — the AI attack surface, the review areas (discovery, network isolation, keyless identity, content safety, RAG exposure, responsible-AI governance), and the boundary vs. the paired AI build solutions and Copilot governance
2. Requirements	Solution-specific prerequisites with Verify / Implement-if-missing
Global Requirements (Assessments)	Category-wide prerequisites, published once
3. Implementation	Exact, ordered, read-only steps — validate access, discover the AI estate, review network isolation, keyless identity, content safety, RAG exposure and secrets, then pilot → validate → expand and produce the risk-ranked report and roadmap

Document	Purpose
5. Testing	Validation gates — read-only proof, complete discovery, portal reconciliation, no preview false positives, reproducible report
6. Troubleshooting	Field issues → cause → fix
7. Best Practices	Field-proven guidance and where to go next (the paired AI solutions)

Automation artifacts (all read-only)

File	What it does
`AIWLSEC-Discovery.ps1`	**PowerShell, read-only.** AI estate discovery across Azure OpenAI / Cognitive Services, AI Foundry, AI Search, and Azure ML — public access, local-auth (keys) state, custom subdomain, model deployments + content-filter policies, private endpoints, diagnostics, and search indexes/data-sources indicating a RAG pattern. Writes per-resource posture to the findings folder.
`AIWLSEC-Validate.ps1`	**PowerShell, read-only.** Pre-flight — confirms PowerShell/module versions, Azure + Graph connected, the identity is Azure Reader + Security Reader (no Contributor/write), AI resource providers registered, and lists the AI resources in scope. The gate that proves the run is read-only and complete.
`AIWLSEC-NetworkIsolation.ps1`	**PowerShell, read-only.** Per-AI-resource network-isolation review — public network access (Enabled = finding), private-endpoint presence, and private-DNS path — the highest-frequency AI findings.
`AIWLSEC-ContentSafety.ps1`	**PowerShell, read-only.** Per-model-deployment content-safety posture — whether a content-filter (RAI) policy is attached and its default state — and flags prompt-shield / jailbreak / groundedness protections as "verify availability" (several are preview). Never asserts a gap for an unavailable feature.
`AIWLSEC-Report.ps1`	**PowerShell, read-only.** Rolls the discovery + network + content-safety outputs (and recorded RAG/secrets findings) into per-resource posture, tags each finding Critical/High/Medium/Low (impact × exposure), names the baseline and run date, flags preview-dependent items, and maps each finding to a paired build solution (AIFOUNDRY/AIRAG/AISAFETY/AILZ).

Plus **read-only KQL** (saved as `AIWorkloadSec – KeyAuth – Signal`) in the Testing guide to corroborate the keyless finding with live authentication evidence where Log Analytics / Sentinel is available.

What the buyer receives

- This assessment guide as a **Word** document and **PDF**.
- All five read-only **PowerShell** scripts plus the KQL, in a single ZIP.
- A repeatable process following the model: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** — entirely

read-only, re-runnable to trend the AI security posture, with remediation handed to the paired AI build solutions under change control.

Delivered as a Complete Solution Package (PDF + Word + scripts).

Outcome

A defensible answer to "is the AI we're building secure?" — an inventory of every Azure AI workload, a risk-ranked list of the prompt-injection, data-leakage, identity, network, and governance gaps that carry the most risk, and a clear hand-off to the paired build solutions — produced by a single administrator, entirely read-only, and re-runnable to prove progress. Acceptable-use, data-classification, and risk-acceptance decisions remain the customer's.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

AIWLSEC-ContentSafety.ps1

Purpose: For each Azure OpenAI model deployment in scope, reports whether a content-filter (RAI) policy is attached and lists the custom RAI policies defined on the account. It also emits a checklist of the higher-order safety controls to VERIFY — prompt shields / jailbreak detection, protected-material detection, and groundedness detection — WITHOUT asserting a gap, because several are in preview or region-limited. The acceptable-content threshold itself is the customer's trust-and-safety decision; this reviews whether controls EXIST, not what they should be set to. Read-only; changes nothing. HOW TO RUN Connect-AzAccount pwsh -File .\AIWLSEC-ContentSafety.ps1 -SubscriptionIds "<sub>" -OutputFolder "<secure path>" Verify the current RAI-policy API surface, property names, and preview status for your region.* EXPECTED RESULTS A per-deployment table (content filter attached? which policy?) and a "verify availability" checklist of preview safety features. CSV in the output folder. Read-only. .NOTES White-label. Read-only. Makes no changes. Preview features are flagged "verify", never asserted as gaps. > [CmdletBinding()] param([string[]]$SubscriptionIds, [Parameter(Mandatory=$true)][string]$OutputFolder)

How to run: Connect-AzAccount pwsh -File .\AIWLSEC-ContentSafety.ps1 -SubscriptionIds "<sub>" -OutputFolder "<secure path>" Verify the current RAI-policy API surface, property names, and preview status for your region.* EXPECTED RESULTS A per-deployment table (content filter attached? which policy?) and a "verify availability" checklist of preview safety features. CSV in the output folder. Read-only. .NOTES White-label. Read-only. Makes no changes. Preview features are flagged "verify", never asserted as gaps. > [CmdletBinding()] param([string[]]$SubscriptionIds, [Parameter(Mandatory=$true)][string]$OutputFolder)

AIWLSEC-Discovery.ps1

Purpose: Builds the point-in-time AI estate baseline. For every Azure OpenAI / Cognitive Services account, Azure AI Search service, and Azure ML workspace in scope it records the risk posture: public network access, local-auth (account keys) enabled, custom subdomain, model deployments and whether a content-filter policy is attached, private endpoints, diagnostics, and (for Search) API-key vs RBAC auth and index/data-source hints of a RAG pattern. Every call is read-only; nothing is changed. HOW TO RUN Connect-AzAccount ; Connect-MgGraph -Scopes "Directory.Read.All" pwsh -File .\AIWLSEC-Discovery.ps1 -SubscriptionIds "<sub>" -OutputFolder "<secure path>" Run AIWLSEC-Validate.ps1 first. *Verify current Az cmdlet/property names and the AI Foundry resource model.* EXPECTED RESULTS A per-resource posture table on screen and CSV/JSON in the output folder. Read-only. .NOTES White-label. Read-only. Makes no changes. Best-effort across evolving API surfaces — verify property names. > [CmdletBinding()] param([string[]]$SubscriptionIds, [Parameter(Mandatory=$true)][string]$OutputFolder)

How to run: Connect-AzAccount ; Connect-MgGraph -Scopes "Directory.Read.All" pwsh -File .\AIWLSEC-Discovery.ps1 -SubscriptionIds "<sub>" -OutputFolder "<secure path>" Run AIWLSEC-Validate.ps1 first. *Verify current Az cmdlet/property names and the AI Foundry resource model.* EXPECTED RESULTS A per-resource posture table on screen and CSV/JSON in the output folder. Read-only. .NOTES White-label. Read-only. Makes no changes. Best-effort across evolving API surfaces — verify property names. > [CmdletBinding()] param([string[]]$SubscriptionIds,

[Parameter(Mandatory=$true)][string]$OutputFolder)

AIWLSEC-NetworkIsolation.ps1

Purpose: For every Azure OpenAI / Cognitive Services, AI Search, and Azure ML resource in scope, reports the network-isolation posture that most often exposes AI workloads: public network access (Enabled = finding), private-endpoint presence, and whether a private-DNS resolution path likely exists. This is where the highest-frequency AI findings live (public-by-default is the norm). Read-only; changes nothing. HOW TO RUN Connect-AzAccount pwsh -File .\AIWLSEC-NetworkIsolation.ps1 -SubscriptionIds "<sub>" -OutputFolder "<secure path>" Requires Az.Accounts / Az.CognitiveServices / Az.Resources / Az.Network / Az.PrivateDns. *Verify versions.* EXPECTED RESULTS A per-resource isolation table (public access / private endpoints / DNS) with a PASS/FINDING verdict, and a CSV in the output folder. Read-only. .NOTES White-label. Read-only. Makes no changes. > [CmdletBinding()] param([string[]]$SubscriptionIds, [Parameter(Mandatory=$true)][string]$OutputFolder)

How to run: Connect-AzAccount pwsh -File .\AIWLSEC-NetworkIsolation.ps1 -SubscriptionIds "<sub>" -OutputFolder "<secure path>" Requires Az.Accounts / Az.CognitiveServices / Az.Resources / Az.Network / Az.PrivateDns. *Verify versions.* EXPECTED RESULTS A per-resource isolation table (public access / private endpoints / DNS) with a PASS/FINDING verdict, and a CSV in the output folder. Read-only. .NOTES White-label. Read-only. Makes no changes. > [CmdletBinding()] param([string[]]$SubscriptionIds, [Parameter(Mandatory=$true)][string]$OutputFolder)

AIWLSEC-Report.ps1

Purpose: Consumes the CSVs produced by AIWLSEC-Discovery.ps1, AIWLSEC-NetworkIsolation.ps1, and AIWLSEC-ContentSafety.ps1 from the input folder and produces a per-resource findings list where each item is tagged Critical/High/Medium/Low (impact x exposure) and mapped to the paired build solution that remediates it (AIFOUNDRY / AIRAG / AISAFETY / AILZ). It reads files only — no tenant calls, changes nothing. Preview-dependent items are carried as "verify", not asserted as hard gaps. HOW TO RUN pwsh -File .\AIWLSEC-Report.ps1 -InputFolder "<findings folder>" -Label "AIWorkloadSec - Estate - Full" -Baseline "Microsoft AI security baseline (verify version)" EXPECTED RESULTS A Markdown findings report and a ranked-findings CSV written to the input folder, plus an on-screen summary. Read-only. .NOTES White-label. Read-only. Reads the review outputs only; makes no changes. > [CmdletBinding()] param([Parameter(Mandatory=$true)][string]$InputFolder, [Parameter(Mandatory=$true)][string]$Label, [string]$Baseline = 'Microsoft AI security baseline + paired-solution target state (verify current version)')

How to run: pwsh -File .\AIWLSEC-Report.ps1 -InputFolder "<findings folder>" -Label "AIWorkloadSec - Estate - Full" -Baseline "Microsoft AI security baseline (verify version)" EXPECTED RESULTS A Markdown findings report and a ranked-findings CSV written to the input folder, plus an on-screen summary. Read-only. .NOTES White-label. Read-only. Reads the review outputs only; makes no changes. > [CmdletBinding()] param([Parameter(Mandatory=$true)][string]$InputFolder, [Parameter(Mandatory=$true)][string]$Label, [string]$Baseline = 'Microsoft AI security baseline + paired-solution target state (verify current version)')

AIWLSEC-Validate.ps1

Purpose: The gate you run before any substantive review. Confirms PowerShell and module versions, that Azure and Microsoft Graph are connected, that the signed-in identity is read-only (Azure Reader + Security Reader with NO Contributor/write role detected), that the AI resource providers are registered,

and lists the AI resources (Azure OpenAI / AI Search / Azure ML) it can see in scope. If this does not pass, a whole review area can silently return empty or the read-only guarantee is broken. HOW TO RUN Connect-AzAccount Connect-MgGraph -Scopes "Directory.Read.All","Policy.Read.All" pwsh -File .\AIWLSEC-Validate.ps1 -SubscriptionIds "<sub1>","<sub2>" Requires PowerShell 7+ and the Az + Microsoft.Graph modules. *Verify current scope names/modules.* EXPECTED RESULTS A PASS/WARN/FAIL checklist and a count of AI resources found. All PASS = ready for AIWLSEC-Discovery.ps1. .NOTES White-label. Read-only. Makes no changes. > [CmdletBinding()] param([string[]]$SubscriptionIds, [string[]]$RequiredGraphScopes = @('Directory.Read.All'))

How to run: Connect-AzAccount Connect-MgGraph -Scopes "Directory.Read.All","Policy.Read.All" pwsh -File .\AIWLSEC-Validate.ps1 -SubscriptionIds "<sub1>","<sub2>" Requires PowerShell 7+ and the Az + Microsoft.Graph modules. *Verify current scope names/modules.* EXPECTED RESULTS A PASS/WARN/FAIL checklist and a count of AI resources found. All PASS = ready for AIWLSEC-Discovery.ps1. .NOTES White-label. Read-only. Makes no changes. > [CmdletBinding()] param([string[]]$SubscriptionIds, [string[]]$RequiredGraphScopes = @('Directory.Read.All'))